TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

LẬP TRÌNH CHO THIẾT BỊ DI ĐỘNG

THÁI NGUYÊN, THÁNG 5 NĂM 2024

Chủ đề 2: Lập trình hướng đối tượng và biểu thức Lambda

Khám phá lập trình hướng đối tượng với Kotlin và cách thiết kế giao diện ứng dụng Android hiệu quả với các nội dung:

- 2.1. Lập trình hướng đối tượng và biểu thức Lambda:

- Lớp và đối tượng
- Hàm khởi tạo
- Kế thừa
- Biểu thức Lambda

- 2.2. Tạo giao diện Android:

- View và ViewGroup
- Layouts cơ bản

Mục tiêu bài học

- Hiểu các khái niệm về lập trình hướng đối tượng như lớp, đối tượng, kế thừa, và hàm khởi tạo.

- Biết cách sử dụng biểu thức Lambda trong Kotlin để viết hàm ngắn gọn, linh hoạt.

- Nắm vững cách thiết kế giao diện cơ bản trong Android với View và Layouts.

Khái niệm lớp

Lớp là gì?

- Một bản thiết kế (blueprint) để tạo các đối tượng.

- Định nghĩa bằng từ khóa class.

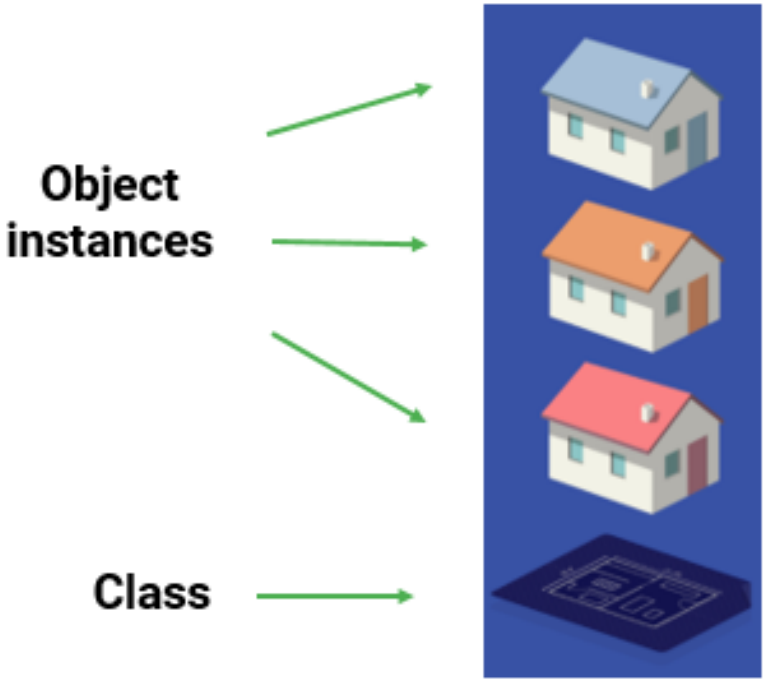
Cấu trúc của lớp:

- Thuộc tính: Đặc điểm của đối tượng.

- Phương thức: Hành vi, hành động.

Ví dụ:

```
class SmartDevice {
    val name = "Android TV"
    fun turnOn() { println("Device is turned on.") }
}
```

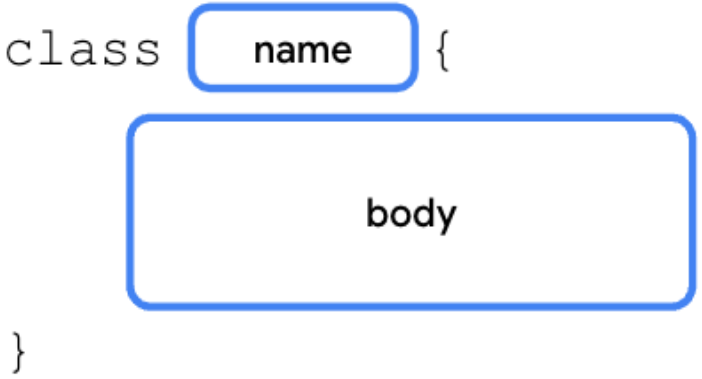


Ví dụ về lớp

Xây dựng lớp SmartDevice.

Cú pháp:

```
class SmartDevice {
    val name = "Android TV"
    fun turnOn() { println("Device is turned on.") }
}
```



Cách sử dụng:

```
val device = SmartDevice()
device.turnOn()
```

Đối tượng là gì?

Khái niệm:

- Đối tượng là một thể hiện của lớp.

- Có trạng thái (thuộc tính) và hành vi (phương thức).

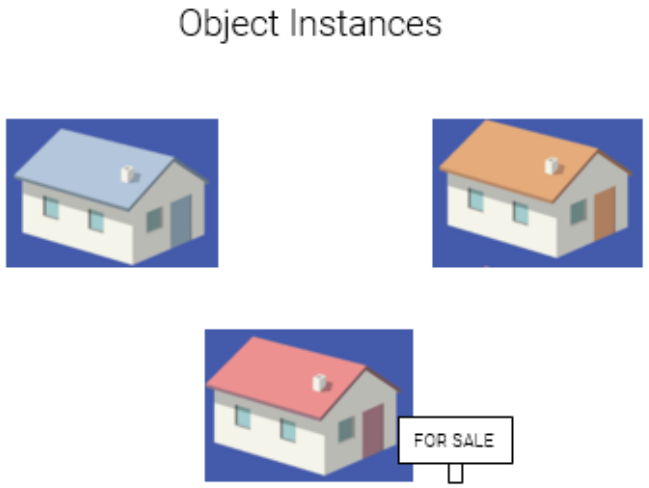
Cú pháp tạo đối tượng:

val device = SmartDevice()

Ví dụ:

- Lớp: SmartDevice

- Đối tượng: device



Thuộc tính và phương thức

- Thuộc tính:

- Các biến lưu trữ dữ liệu của đối tượng.
- Khai báo bằng val hoặc var.

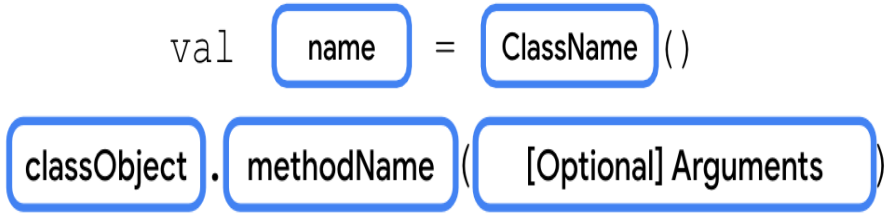
- Phương thức:

- Các hàm biểu diễn hành vi của đối tượng.
- Khai báo bằng fun.

Thuộc tính và phương thức

Ví dụ:

```
class Car(val color: String) {
    fun drive() { println("Driving a $color car!") }
}
val car = Car("red")
car.drive()
```



Thực hành

- Viết lớp Car với:
  - Thuộc tính: color, brand.
  - Phương thức: start(), stop().

Ví dụ mẫu:

```
class Car(val color: String, val brand: String) {
    fun start() { println("$brand car is starting.") }
    fun stop() { println("$brand car is stopping.") }
}
```

```
val myCar = Car("blue", "Toyota")
myCar.start()
myCar.stop()
```

Get và set

set và get là các phương thức truy cập thuộc tính trong Kotlin.Chúng thường được sử dụng để đóng gói (encapsulation) và kiểm soát việc truy cập dữ liệu.

- **get (Truy xuất giá trị):**

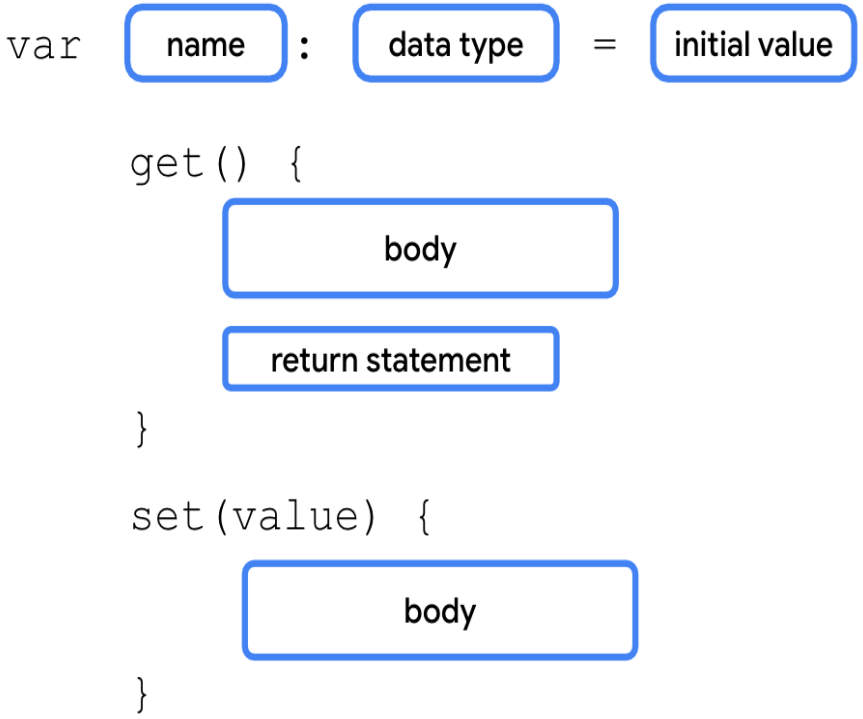
```
var name: String = "Nam"
    get() = field.uppercase()
```

Lấy giá trị của name và chuyển thành chữ hoa.

set (Thiết lập giá trị):

```
var age: Int = 0
set(value) {
    field = if (value > 0) value else 0
}
```

Đảm bảo age không âm khi gán giá trị.



Lợi ích khi sử dụng get, set: Bảo vệ dữ liệu thông qua kiểm tra đầu vào.Tùy chỉnh cách lưu và lấy dữ liệu.

Ví dụ: Get và set

Giải thích:

- field: Biến ẩn chứa giá trị của thuộc tính.
- Getter (get()): Lấy giá trị thuộc tính.
- Setter (set()): Cập nhật giá trị thuộc tính với logic tùy chỉnh.

Lợi ích:

- Đóng gói dữ liệu: Bảo vệ thuộc tính tránh các giá trị không hợp lệ.
- Tùy chỉnh logic: Dễ dàng thêm các kiểm tra trước khi lưu hoặc lấy dữ liệu.

```
class Person {
    var name: String = "Không xác định"
        get() = field
        set(value) {
            field = value.uppercase()
        }
}
fun main() {
    val p = Person()
    p.name = "Nam"
    println(p.name)  // Output: NAM
}
```

Hàm khởi tạo

Hàm khởi tạo là gì?

- Là hàm đặc biệt để khởi tạo đối tượng.

- Có hai loại: khởi tạo chính và khởi tạo phụ.

Hàm khởi tạo mặc định

Hàm khởi tạo mặc định là hàm khởi tạo không có tham số:

```
class SmartDevice constructor() {

}
```

Kotlin luôn hướng đến sự súc tích, vì vậy có thể viết như sau:

```
class SmartDevice {

}
```

Trình biên dịch Kotlin sẽ tự động tạo hàm khởi tạo mặc định.

Hàm khởi tạo chính

Một lớp chỉ có thể có một hàm khởi tạo chính, được xác định là một phần của tiêu đề lớp. Một hàm khởi tạo chính có thể là một hàm khởi tạo mặc định hoặc có tham số. Hàm khởi tạo chính không có nội dung. Điều đó có nghĩa là nó không được chứa bất kỳ mã nào.

```
class Device(val name: String, val type: String)

val device = Device("Smartphone", "Android")
```

ClassName ([Optional] constructor arguments)

Hàm khởi tạo phụ

Một lớp có thể có nhiều hàm khởi tạo phụ.

Hàm khởi tạo phụ có hoặc không có tham số.

```
class name constructor( parameters ) {
    body
}
```

```
class Device(val name: String) {
    constructor(name: String, type: String) : this(name) {
        println("Device type: $type")
    }
}
val device = Device("Smartphone", "Android")
```

Kết quả: In ra Device type: Android.

So sánh giữa hai loại hàm khởi tạo

- Khởi tạo chính: Đơn giản và gọn gàng hơn.
- Khởi tạo phụ: Linh hoạt, có thể thêm logic khởi tạo.

Thực hành

Viết một lớp Book với:

- Thuộc tính: title, author.

- Hai hàm khởi tạo: chính và phụ.

Kế thừa là gì?

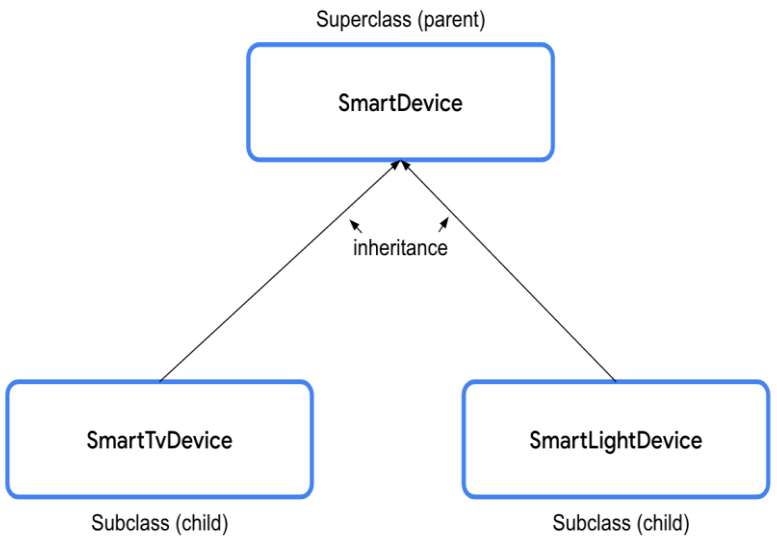
Kế thừa cho phép một lớp mở rộng từ lớp khác.

Lớp cha chứa các đặc điểm và hành vi chung.

Lớp con kế thừa và mở rộng thêm.

```
class Subclass name ( [optional] parameters ) :
      Superclass name ( [optional] parameters ) {

      body
}
```



Ví dụ về kế thừa

```
open class Animal {
    fun eat() { println("Animal is eating") }
}
class Dog : Animal() {
    fun bark() { println("Dog is barking") }
}
```

Kết quả: Dog có thể gọi cả eat và bark.

Trong Kotlin, theo mặc định, tất cả các lớp đều là cuối cùng, nghĩa là không thể mở rộng chúng, vì vậy phải xác định mối quan hệ giữa các lớp.

Từ khoá open thông báo cho trình biên dịch rằng lớp/hàm/thuộc tính này có thể mở rộng được.

Ghi đè phương thức

Khi lớp con cần thay đổi cách hoạt động của lớp cha

- Dùng từ khóa override.

Ví dụ:

```
open fun makeSound() { println("Animal sound") }

override fun makeSound() { println("Dog barks") }
```

super.functionName ([Optional] Arguments)

Ghi đè thuộc tính

Tương tự như phương thức, cũng có thể ghi đè các thuộc tính theo các bước tương tự.

```
open class SmartDevice(val name: String, val category: String) {
    var deviceStatus = "online"
    open val deviceType = "unknown"
    ...
}

class SmartTvDevice(deviceName: String, deviceCategory: String) :
    SmartDevice(name = deviceName, category = deviceCategory) {
    override val deviceType = "Smart TV"
    ...
}
```

Hoạt động thực hành

Tạo lớp Animal với phương thức speak().

Tạo lớp con Dog, ghi đè phương thức speak() với nội dung "Woof!".

So sánh với giao diện

- Kế thừa: Chỉ một lớp cha.
- Giao diện: Có thể triển khai nhiều giao diện.
- Dùng giao diện khi không cần chia sẻ logic giữa các lớp.

Biểu thức Lambda là gì?

Lambda là một hàm ẩn danh, không có tên.

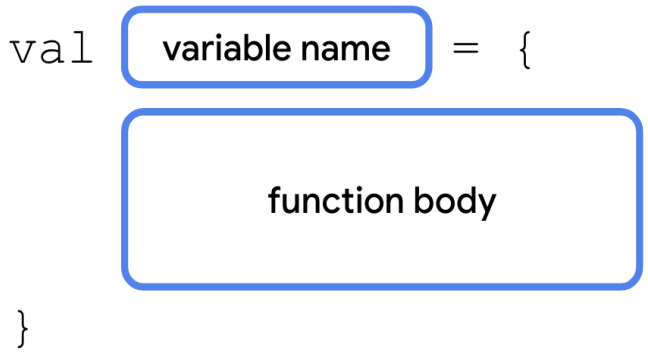
Sử dụng khi cần định nghĩa các hàm ngắn gọn, thường trong xử lý sự kiện.

Đặc điểm:

- Ngắn gọn hơn hàm thông thường.
- Có thể truyền như một giá trị vào hàm khác.

Cú pháp cơ bản:

- { thamSo -> bieuThuc }



Ví dụ cơ bản về Lambda

Ví dụ minh họa:

```
val greet = { println("Hello!") }

greet() // Kết quả: In ra "Hello!"
```

Lambda không cần tên hàm, giúp mã ngắn gọn hơn.

Sử dụng Lambda với tham số

Cú pháp với tham số:

- { thamSo1, thamSo2 -> bieuThuc }

Ví dụ:

```
val sum = { x: Int, y: Int -> x + y }
println(sum(3, 5)) // Kết quả: 8
```

Tối ưu cú pháp Lambda

Nếu chỉ có một tham số, có thể dùng từ khóa it thay vì đặt tên tham số.

Ví dụ:

```
val square = { it * it }

println(square(4)) // Kết quả: 16
```

Ứng dụng Lambda trong xử lý sự kiện

Tình huống: Xử lý nhấn nút trong ứng dụng Android.

Ví dụ:

```
button.setOnClickListener {
    println("Button clicked!")
}
```

Lambda giúp mã xử lý sự kiện ngắn gọn và dễ đọc hơn.

So sánh Lambda và hàm thông thường

Lambda:

- Gọn gàng, dễ viết.

- Không cần tên hàm.

- Sử dụng trong các tác vụ nhỏ.

Hàm thông thường:

- Có thể tái sử dụng.

- Tốt hơn cho logic phức tạp.

Khi nào dùng Lambda?

- Khi xử lý các tác vụ đơn giản như sự kiện, lặp.

Thực hành

Nhiệm vụ: Viết Lambda để kiểm tra số chẵn lẻ.

Yêu cầu:

- Truyền một số nguyên vào Lambda.

- In ra "Even" nếu số là chẵn, "Odd" nếu là lẻ.

Gợi ý:

```
val checkEvenOdd = { n: Int ->
    if (n % 2 == 0) "Even" else "Odd"
}
println(checkEvenOdd(3))
```

Dùng hàm làm loại dữ liệu trả về

Hàm là loại dữ liệu, vì vậy có thể dùng hàm như bất kỳ loại dữ liệu nào khác.

Thậm chí có thể trả về hàm từ hàm khác.

```
fun [function name]() : [function type] {
    // code
    return [Name of another function]
}
```

```
fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

// Sử dụng lambda để truyền vào hàm
fun main() {
    val sum = calculate(5, 3) { x, y -> x + y }
    println("Tổng: $sum")  // Output: Tổng: 8

    val multiply = calculate(4, 2) { x, y -> x * y }
    println("Tích: $multiply")  // Output: Tích: 8
}
```

Tóm tắt về Lambda

Lambda là một trong những tính năng mạnh mẽ nhất của Kotlin.

Giúp mã ngắn gọn, dễ đọc.

Ứng dụng trong:

- Xử lý sự kiện.

- Làm việc với danh sách (map, filter, reduce).

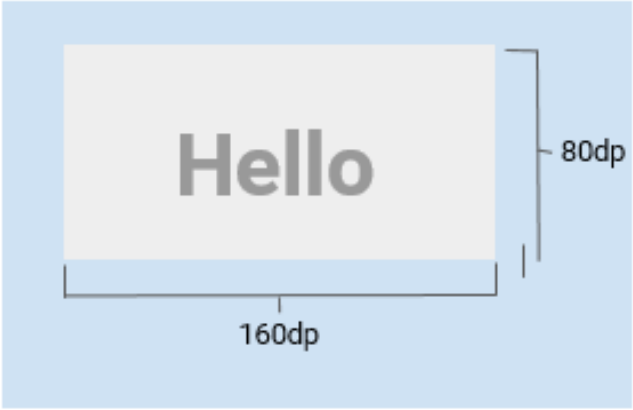
- Truyền logic vào các hàm cao cấp.

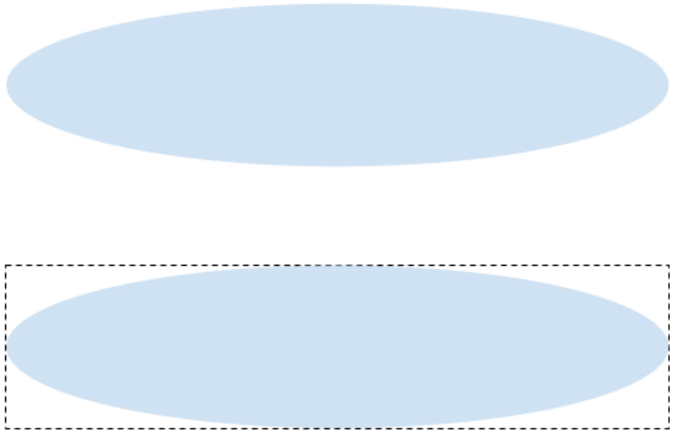
View là gì?

View là thành phần cơ bản trong giao diện Android.

- Trách nhiệm: Vẽ nội dung lên màn hình và xử lý sự kiện người dùng.

- Ví dụ: TextView, Button, ImageView.





Các dạng kích cỡ màn hình thường gặp

Density qualifier	Description	DPI estimate
ldpi (mostly unused)	Low density	~120dpi
mdpi (baseline density)	Medium density	~160dpi
hdpi	High density	~240dpi
xhdpi	Extra-high density	~320dpi
xxhdpi	Extra-extra-high density	~480dpi
xxxhdpi	Extra-extra-extra-high density	~640dpi

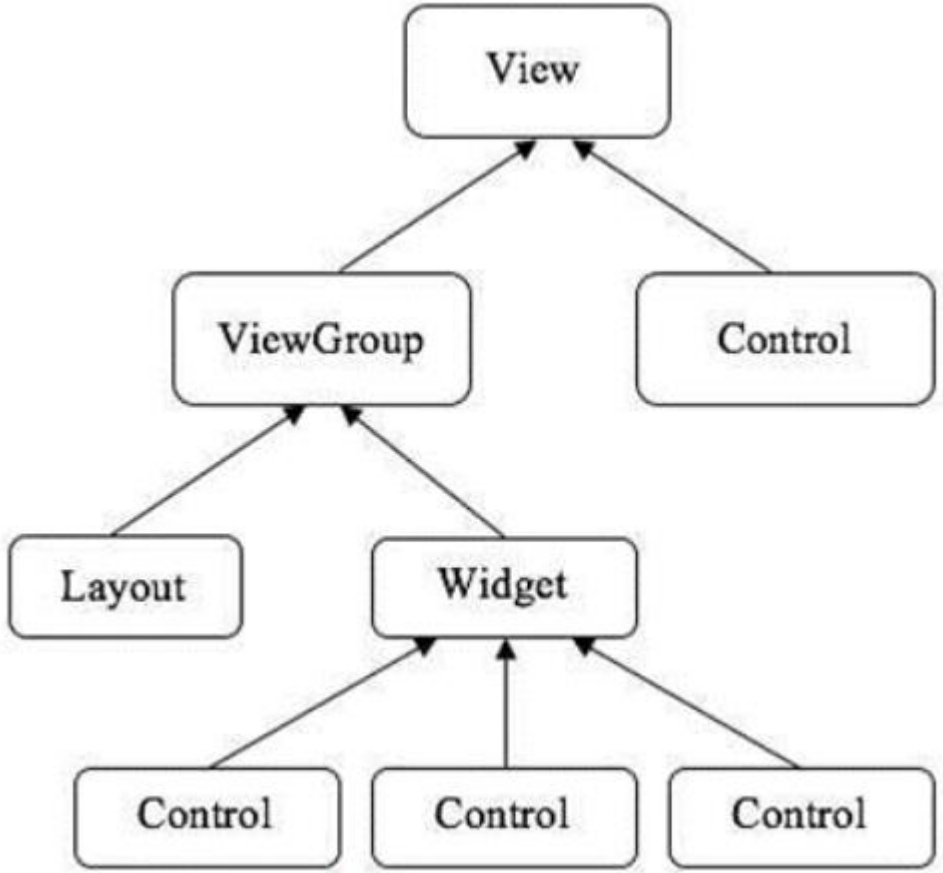
ViewGroup là gì?

ViewGroup là container chứa các View con.

- Chịu trách nhiệm sắp xếp các View con.

- Các loại phổ biến:

- LinearLayout
- FrameLayout
- ConstraintLayout



Giới thiệu về FrameLayout

FrameLayout là một loại layout trong Android.

- Đặt các View chồng lên nhau.

- Phù hợp cho giao diện đơn giản hoặc lớp nền (background layer).

Hướng dẫn sử dụng FrameLayout

Các thuộc tính quan trọng:

- layout_gravity: Đặt vị trí của View trong FrameLayout.

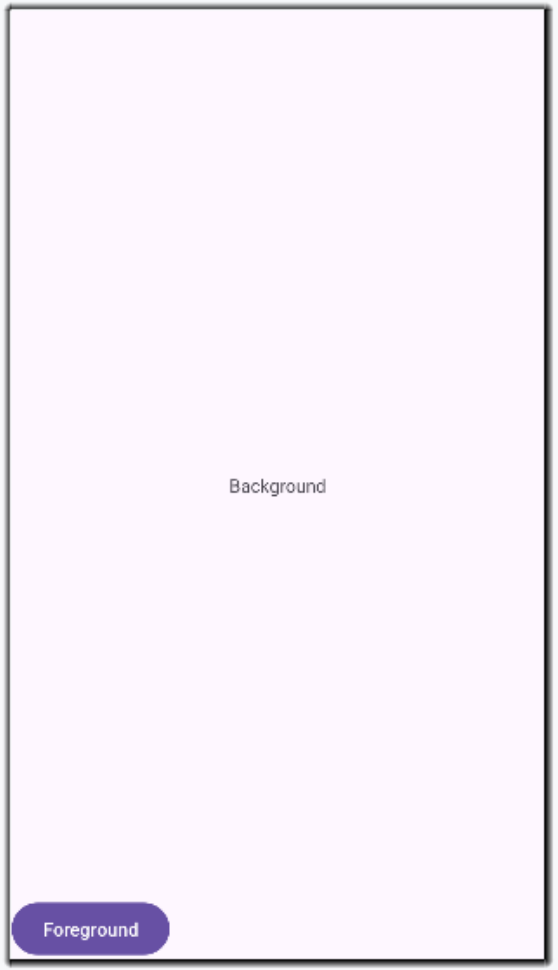
- padding: Tạo khoảng cách bên trong.

- margin: Tạo khoảng cách bên ngoài.

Thường sử dụng cho giao diện đơn giản.

Ví dụ về FrameLayout

```xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:android="http://schemas.android.com/apk/res/android" >
    <TextView android:text="Background"
        android:layout_gravity="center"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"/>
    <Button
        android:text="Foreground"
        android:layout_gravity="bottom"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</FrameLayout>
```



Giới thiệu về LinearLayout

LinearLayout sắp xếp các View con theo chiều dọc hoặc ngang.

- Sử dụng orientation để chỉ định hướng.

- Phù hợp cho giao diện tuyến tính đơn giản.

Hướng dẫn sử dụng LinearLayout

Các thuộc tính quan trọng:

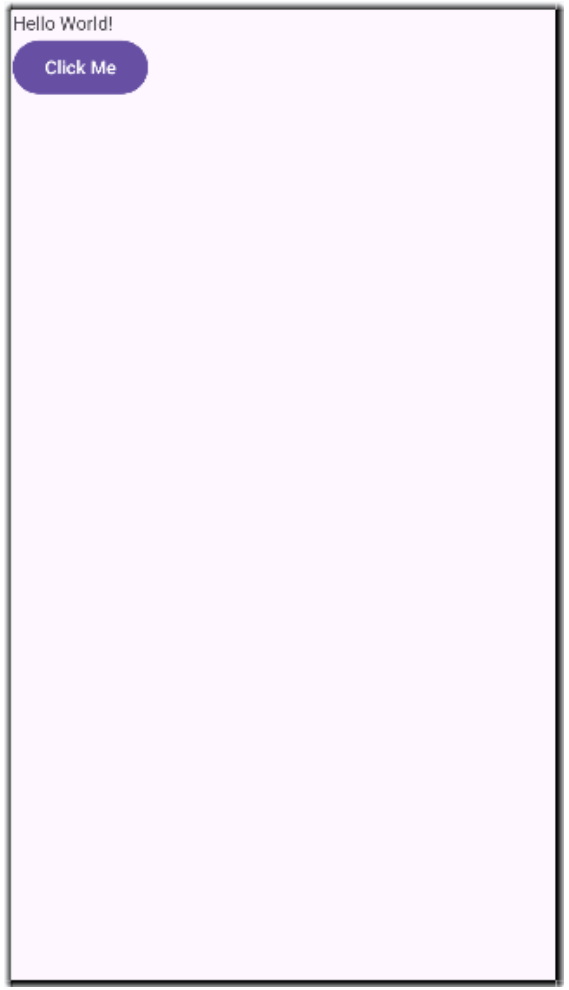
- orientation: Hướng sắp xếp (vertical hoặc horizontal).

- weight: Phân phối không gian giữa các View con.

- gravity: Căn chỉnh nội dung.

Ví dụ về LinearLayout

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <TextView
        android:text="Hello World!"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
    <Button
        android:text="Click Me"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        />
</LinearLayout>
```

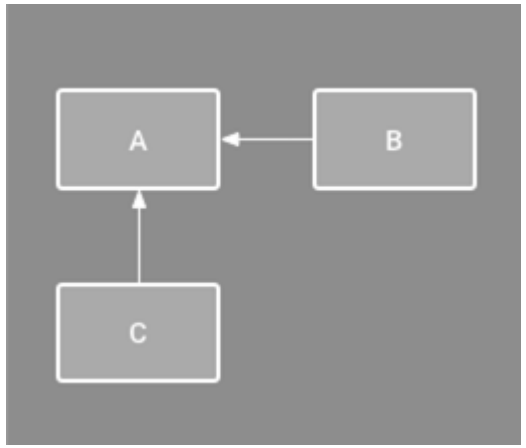


Giới thiệu về ConstraintLayout

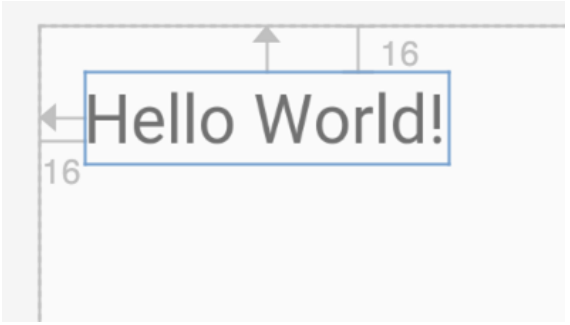
ConstraintLayout là layout mạnh mẽ và linh hoạt.

- Cho phép tạo giao diện phức tạp.

- Sử dụng các ràng buộc (constraints) để định vị View.




```
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
```



Hướng dẫn sử dụng ConstraintLayout

Các thuộc tính quan trọng:

- layout_constraintTop_toTopOf: Ràng buộc phía trên.

- layout_constraintBottom_toBottomOf: Ràng buộc phía trên.

- layout_constraintStart_toStartOf: Ràng buộc phía trái.

- layout_constraintEnd_toEndOf: Ràng buộc phía phải.

Hỗ trợ tạo giao diện gọn gàng, dễ bảo trì.

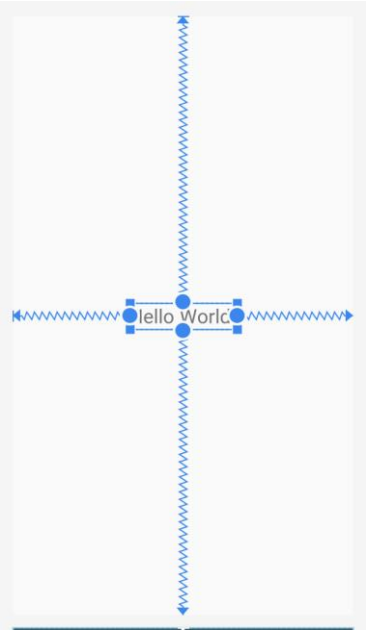
Ví dụ về ConstraintLayout

Ví dụ XML:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        ...
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

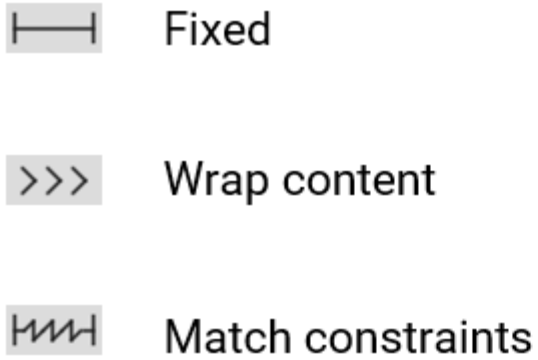


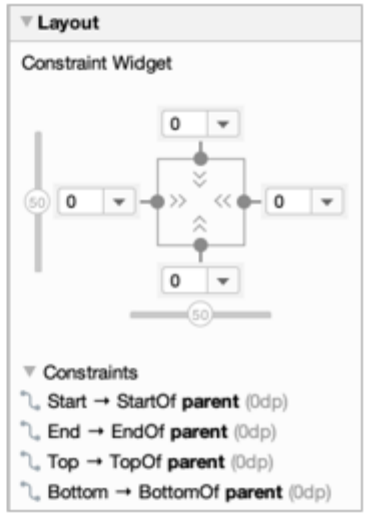
Thực hành với Layouts

Nhiệm vụ, sử dụng công cụ hỗ trợ để:

- Tạo giao diện đơn giản gồm một TextView và hai Button.

- Sử dụng ConstraintLayout để sắp xếp các thành phần.





Câu hỏi thảo luận

1. So sánh LinearLayout và ConstraintLayout.

2. Điểm khác biệt giữa Lambda và hàm thông thường trong Kotlin?

Bài tập ứng dụng

1. Tạo giao diện với ConstraintLayout gồm các thành phần: TextView, Button, ImageView.

2. Viết Lambda để kiểm tra các số nguyên tố từ 1 .. 100

Tóm tắt bài học

Nội dung đã học:

- Lập trình hướng đối tượng: lớp, đối tượng, kế thừa.

- Biểu thức Lambda: gọn gàng, linh hoạt.

- Giao diện Android: View, ViewGroup, Layouts.

Câu hỏi ôn tập

- Hàm: Khai báo hàm, hàm với tham số mặc định
- Tạo lớp dữ liệu
- Biểu thức Lambda, cách khai báo một biểu thức, truyền lambda cho hàm
- Lớp, cách tạo lớp phương thức get, set, lớp kế thừa, phương thức khởi dựng

Bài tập thực hành

1. Tạo lớp SmartDevice với các thuộc tính name, category và các phương thức như turnOn() và turnOff().

2. Viết chương trình sử dụng biểu thức Lambda để lọc danh sách các số nguyên chẵn từ 1 đến 10.

3. Tạo lớp Car với các thuộc tính brand, model, price. Thêm phương thức hiển thị thông tin xe.

4. Viết chương trình sử dụng biểu thức Lambda để lọc danh sách các chuỗi có độ dài lớn hơn 5.

5. Sử dụng kế thừa để tạo lớp con ElectricCar kế thừa từ lớp Car và thêm thuộc tính batteryCapacity.

6. Sử dụng biểu thức Lambda để sắp xếp danh sách số nguyên giảm dần.

Bài tập thực hành

7. Mục tiêu: Tạo giao diện đăng nhập đơn giản: Sử dụng LinearLayout theo hướng dọc, có hai EditText cho tên người dùng và mật khẩu, nút Button với dòng chữ "Đăng nhập".

8. Tạo giao diện ứng dụng máy tính cơ bản: Sử dụng GridLayout hoặc TableLayout, tạo các nút số từ 0-9 và các nút phép tính: +, -, *, /, =, kết quả hiển thị trong TextView phía trên.

9. Mục tiêu: Tạo giao diện hồ sơ cá nhân với ảnh đại diện: Sử dụng RelativeLayout hoặc ConstraintLayout, hiển thị ảnh đại diện với ImageView, hiển thị tên và mô tả ngắn gọn bằng TextView.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024